Toán học của Gradient Ascent trên Pixel

Trong huấn luyện thông thường (Gradient Descent): Cố định Dữ liệu, **Cập nhật Trọng số (Weights)** để giảm thiểu Hàm Loss về cực tiểu.

$$W_{new} = W_{old} - \alpha \cdot \nabla_W L(W) \tag{1}$$

Trong Trực quan hóa Bộ lọc (Gradient Ascent): Cố định Trọng số, **Cập nhật Pixel của Ảnh (X)** để **tối đa hóa** Giá trị Kích hoạt của Bộ lọc (Activation A).

$$X_{new} = X_{old} + \alpha \cdot \nabla_X A(X) \tag{2}$$

Ta tính Đạo hàm của Đầu ra Bộ lọc theo Đầu vào Ảnh, sau đó cộng thêm đạo hàm đó vào Ảnh để làm nó rõ nét hơn qua từng epoch.

Định nghĩa Hàm Mất mát (Loss) cho Bộ lọc

Ta dùng cấu trúc Xception siêu mạnh để thực hành kỹ thuật này. Hãy định nghĩa một hàm `compute_loss` trả về trung bình giá trị kích hoạt của 1 channel.

```
import keras_hub
from keras import ops

model = keras_hub.models.Backbone.from_preset("xception_41_imagenet")
layer = model.get_layer(name="block3_sepconv1")
feature_extractor = keras.Model(inputs=model.input, outputs=layer.output)

def compute_loss(image, filter_index):
    activation = feature_extractor(image)
    # We avoid border artifacts by discarding the first 2 pixels
    filter_activation = activation[:, 2:-2, 2:-2, filter_index]
    # Returns the mean of the activation values for the filter
    return ops.mean(filter_activation)
```

Vòng lặp Gradient Ascent (TensorFlow)

Sử dụng `tf.GradientTape` để tính toán Gradient của Loss theo Image.

```python
import tensorflow as tf

@tf.function
def gradient_ascent_step(image, filter_index, learning_rate):
    with tf.GradientTape() as tape:
        # Explicitly watches the image tensor
        tape.watch(image)
        loss = compute_loss(image, filter_index)

    # Computes the gradients of the loss with respect to the image
    grads = tape.gradient(loss, image)
    # Applies the "gradient normalization trick"
    grads = ops.normalize(grads)
    # Moves the image a little bit in a direction that activates it
    image += learning_rate * grads
    return image
```

Khởi tạo Mảng Nhiễu Trắng ban đầu

Viết vòng lặp lặp lại 30 bước. Tại thời điểm t=0, bức ảnh là một mảng nhiễu trắng vô nghĩa.

```
def generate_filter_pattern(filter_index):
    iterations = 30
    learning_rate = 10.0
    image = keras.random.uniform(
        # Initialize an image tensor with random values centered on 0.5
        minval=0.4, maxval=0.6, shape=(1, 200, 200, 3)
    )

    # Repeatedly updates the values of the image tensor
    for i in range(iterations):
        image = gradient_ascent_step(image, filter_index, learning_rate)

    return image[0]
```

Xử lý hậu kỳ ảnh (Deprocess Image)

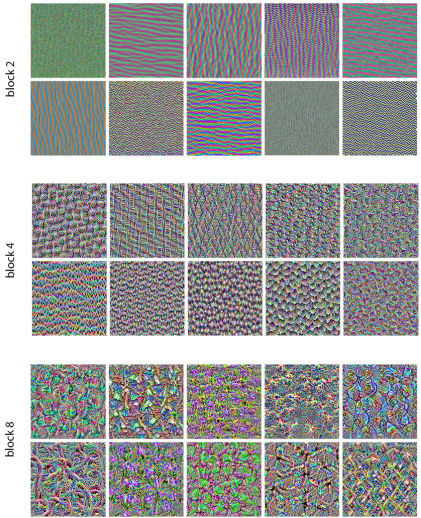
Sau vòng lặp 30 bước, ma trận thu được chứa các giá trị float rất dị thường. Ta phải ép nó về khoảng 0–255 chuẩn RGB.

```
def deprocess_image(image):
    # Normalizes image values
    image -= ops.mean(image)
    image /= ops.std(image)
    image *= 64
    image += 128
    image = ops.clip(image, 0, 255)

    # Center crop to avoid border artifacts
    image = image[25:-25, 25:-25, :]
    image = ops.cast(image, dtype="uint8")
    return ops.convert_to_numpy(image)
```

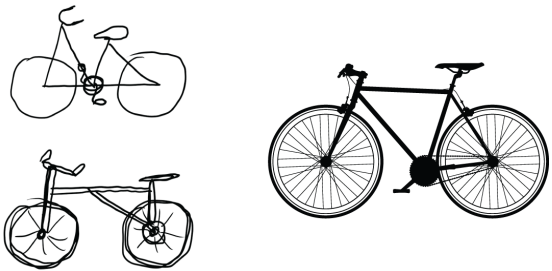
Quần thể các Mẫu Đặc trưng lý tưởng

- Kết quả của Gradient Ascent là những họa tiết kỳ ảo giống như ảo giác (Psychedelic).
- Các lớp nông (Shallow): Học màu sắc, vân gỗ, sóng nước.
- Các lớp giữa (Middle): Học kết cấu phức tạp hơn (chấm bi, sọc vằn, mạng nhện).



Lớp Sâu - Các Khái niệm Đời thực

- Đối với các khối ở cực sâu (như `block5_conv1` của VGG16/Xception), các họa tiết bắt đầu mang hình thù của vật thể thực.
- Chẳng hạn, bộ lọc này tổng hợp nên họa tiết hình elip đan chéo, rất giống nan hoa xe đạp (Bicycles), lông chim vũ hay vảy bò sát.
- Đây là bằng chứng tuyệt vời cho thấy Convolution Layers hoàn toàn có thể hiểu cấu trúc của vạn vật!



Hình 10.6: Họa tiết bộ lọc siêu sâu (Xe đạp / Vảy)

Kỹ thuật 3: Class Activation Map (CAM)

- **Mục tiêu:** Trả lời câu hỏi "Phần nào của bức ảnh đã khiến Mạng CNN đưa ra quyết định phân loại này?".
- Rất hữu ích cho **Phân tích lỗi (Error Analysis)** (Tại sao nhận diện sai?) và **Khám phá Y khoa** (Chỉ ra khối u nằm ở đâu trên ảnh X-Quang phổi).
- Phương pháp SOTA (State-of-the-art) là **Grad-CAM** (Gradient-weighted Class Activation Mapping).

Toán học của Grad-CAM

1. Tính Đạo hàm của xác suất Lớp dự đoán (Vd: African elephant y^c) theo toàn bộ Bản đồ đặc trưng (Feature Maps A^k) ở Lớp Tích chập **cuối cùng**.
2. Lấy Trung bình không gian (Global Average Pooling) của Đạo hàm này để ra Trọng số độ quan trọng α_k^c của Kênh k:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{i,j}^k} \tag{3}$$

3. Nhân chập Trọng số với Feature Maps tương ứng, rồi bọc qua hàm ReLU (chỉ lấy phần dương - Positive influence):

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k\right) \tag{4}$$

Đầu vào bài toán Phân tích Lỗi

- Ta đưa bức ảnh chụp hai con Voi châu Phi và con cái của chúng vào mô hình Xception chuẩn.
- Lớp `ImageClassifier` dự đoán: Voi châu Phi (African elephant) với xác suất cực cao.
- Câu hỏi đặt ra: Nó nhìn vào Voi mẹ hay Voi con để ra quyết định?



Hình 10.7: Ảnh Voi châu Phi đầu vào

Chuẩn bị Grad-CAM: Trích xuất Lớp Cuối

Ta tạo 2 Model phụ: Model 1 lấy đầu ra của Lớp tích chập cuối cùng. Model 2 chạy từ đầu ra đó đến thẳng lớp Softmax dự đoán.

```
# Model 1: From original image to the last conv layer feature map
last_conv_layer_name = "block14_sepconv2_act"
last_conv_layer = model.backbone.get_layer(last_conv_layer_name)
last_conv_layer_model = keras.Model(model.inputs, last_conv_layer.output)

# Model 2: From last conv layer feature map to predictions
classifier_input = last_conv_layer.output
x = classifier_input
for layer_name in ["pooler", "predictions"]:
    x = model.get_layer(layer_name)(x)
classifier_model = keras.Model(classifier_input, x)
```

Đạo hàm Xếp hạng Lớp (Class Gradients)

Dùng `tf.GradientTape` để lấy Gradient của Node dự đoán "African Elephant" đối chiếu với Bản đồ đặc trưng lớp dưới.

```
def get_top_class_gradients(img_array):
    last_conv_layer_output = last_conv_layer_model(img_array)
    with tf.GradientTape() as tape:
        tape.watch(last_conv_layer_output)
        preds = classifier_model(last_conv_layer_output)
        top_pred_index = ops.argmax(preds[0])
        top_class_channel = preds[:, top_pred_index]

    # Gradient of top class w.r.t the feature map
    grads = tape.gradient(top_class_channel, last_conv_layer_output)
    return grads, last_conv_layer_output

grads, last_conv_output = get_top_class_gradients(img_array)
```

Xây dựng Bản đồ Nhiệt (Heatmap)

Nhân chập Gradient (Trọng số tầm quan trọng của kênh) vào Feature Map để vẽ ra Heatmap thô.

```python
import numpy as np
import matplotlib.pyplot as plt

# Global average pooling the gradients
pooled_grads = np.mean(grads, axis=(0, 1, 2))
last_conv_output = last_conv_output[0].copy()

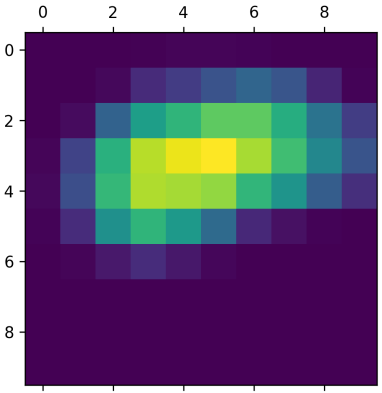
# Multiplies each channel by how important it is
for i in range(pooled_grads.shape[-1]):
    last_conv_output[:, :, i] *= pooled_grads[i]

# The channel-wise mean is our heatmap
heatmap = np.mean(last_conv_output, axis=-1)

# Apply ReLU manually
heatmap = np.maximum(heatmap, 0)
heatmap /= np.max(heatmap)
```

Bản đồ Nhiệt thô Grad-CAM

- Kết quả là một mảng Numpy 2D.
- Điểm có giá trị càng cao (Vùng sáng vàng chói) thì càng tác động lớn đến quyết định phân loại "Đây là con voi".



Hình 10.8: Bản đồ nhiệt thô Grad-CAM

Chồng bản đồ nhiệt (Superimposed Heatmap)

Ta áp dụng phổ màu JET (Xanh dương sang Đỏ) của `matplotlib` để nhuộm màu bản đồ, rồi áp lên trên ảnh gốc với hệ số trong suốt 0.4.

```
import matplotlib.cm as cm

img = keras.utils.load_img(img_path)
img = keras.utils.img_to_array(img)

heatmap = np.uint8(255 * heatmap)
jet = cm.get_cmap("jet")
jet_colors = jet(np.arange(256))[:, :3]
jet_heatmap = jet_colors[heatmap]

jet_heatmap = keras.utils.array_to_img(jet_heatmap)
jet_heatmap = jet_heatmap.resize((img.shape[1], img.shape[0]))
jet_heatmap = keras.utils.img_to_array(jet_heatmap)

# Superimposes the heatmap at 40% opacity
superimposed_img = jet_heatmap * 0.4 + img
```